SET 14

PART A — Short questions and answers

Q1. Classify the attributes Farmer_ID, Phone_Numbers and Age of a FARMER as key, multivalued or derived.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Classify the commands CREATE, INSERT, UPDATE and ALTER as DDL or DML.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain how the dependency Village → District makes a FARMER relation violate 3NF.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain the purpose of the DISTINCT keyword when listing crop names.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain the use of ROUND with AVG to display the average yield.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Explain the role of the %ROWTYPE attribute.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Describe how the primary key of a relational table corresponds to _id in MongoDB.

Answer: A relational table stores rows and columns with a fixed schema. A MongoDB collection stores flexible BSON documents.

Q8. Describe the use of the use command in mongosh to switch to a database.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Explain what the following MongoDB query returns: db.farmers.find({Village: "Gudur", Crops: "Paddy"})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Describe the output of: MATCH (c:Crop) RETURN c.name ORDER BY c.msp DESC LIMIT 3;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An agricultural cooperative maintains a Farm Management System to store farmer details, crop details,

1(a) ER design:

• **FARMER**(FarmerID, FarmerName, Village, Phone); primary key: **FarmerID**.

• **CROP**(CropID, CropName, Season, MSP); primary key: **CropID**.

• **HARVEST**(HarvestID, FarmerID, CropID, HarvestDate, YieldQuintals); primary key: **HarvestID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE FARMER (
  FarmerID NUMBER PRIMARY KEY,
  FarmerName VARCHAR2(100),
  Village VARCHAR2(100),
  Phone VARCHAR2(100)
```

```
);
CREATE TABLE CROP (
  CropID NUMBER PRIMARY KEY,
  CropName VARCHAR2(100),
  Season VARCHAR2(100),
  MSP VARCHAR2(100)
);
CREATE TABLE HARVEST (
  HarvestID NUMBER PRIMARY KEY,
  FarmerID NUMBER,
  CropID NUMBER,
  HarvestDate DATE,
  YieldQuintals NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM HARVEST x
JOIN FARMER a ON x.FarmerID = a.FarmerID
JOIN CROP b ON x.CropID = b.CropID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```